January 2026

HSLU

Reimplementation of clickstream event sequence analysis model TRACE

Author:

Manuel Fas

Supervisor:

Jan Svoboda

GitHub Repository:

https://github.com/Fasmanuel2002/AAI_HSLU_TRACE

Table of Contents

Abstract **3**

Introduction **4**

Project Overview **4**

Literature Review **6**

Available Datasets **10**

Methodology **18**

Experiments and Results **27**

Conclusion **33**

References **35**

Abstract

On e-commerce websites, users often follow complex navigation paths while searching for products and ultimately completing a purchase. These navigation patterns generate clickstream data that captures detailed information about user interactions over time and across sessions. Such data can be leveraged to enhance personalization and improve recommendation systems by more effectively modeling user behavior and intent.

This research presents a re-implementation and investigation of the *TRACE* architecture, a novel transformer-based approach designed to generate rich user embeddings from multi-session clickstream data for real-world recommendation applications. The study evaluates the effectiveness of *TRACE* in modeling complex user navigation patterns using alternative large-scale e-commerce datasets, due to the unavailability of the original dataset used in the *TRACE* paper.

Introduction

On products e-commerce websites users often exhibit complex navigation patterns while browsing products options before making purchase. A typical user may land on the homepage, search for a product, and leave the platform, only to return several days later to browse again and eventually complete a purchase.

The resulting clickstream data captures these complex user journeys and provides valuable insights into users' behavior and intentions. By making use of this data and gaining a deeper understanding of user behavior, e-commerce platforms can significantly improve personalization by matching users with more relevant content and adapting the experience to their current context. For example, users in the exploration phase can be presented with more suitable products aligned with their interests and timing [1].

However, achieving this level of personalization is challenging, as user journeys often span multiple sessions over extended periods of time, and the target event (such as making a purchase) may occur infrequently within this temporal window.

Project Overview

This research project re-implements the *Transformed-based user Representation from Attributed Clickstream Event Sequence (TRACE)* architecture using PyTorch and evaluates it on alternative datasets, as the original dataset used in the paper is not publicly available. TRACE proposes a novel approach for learning rich user representations from multi-session clickstream data with sparse target signals.

The *TRACE* architecture is built around a multi-task learning (MTL) framework, in which a shared Transformer encoder is trained to predict multiple user engagement objectives from sequences of clickstream events. By predicting a diverse set of future user engagement signals, the architecture is encouraged to learn robust and versatile user representations that generalize across tasks and sessions [1].

The main goal of this research project is to re-implement the *TRACE* architecture using alternative datasets and to evaluate whether *TRACE* is effective at learning from and predicting user behavior based on attributed clickstream event sequences across datasets that differ from the original TRACE dataset.

Potential Applications of this Research

- **Product E-commerce websites***:*

  The proposed approach can be applied to product clickstream event in product e-commerce platforms. In particular, the experiments conducted in this project make use of data from **OTTO**, a Swiss retail company that offers a wide range of products, including food, furniture, sporting goods, and cosmetics. The clickstream data captures user interactions across multiple sessions, making it well suited for evaluating the ability of *TRACE* to model complex user behavior and long-term engagement patterns.

- **Travel E-commerce websites:**

  TRACE was originally designed for tourism and travel-related websites. Based on the scope of this research, the *TRACE* architecture could be applied to a wide range of travel-oriented e-commerce platforms.

Literature Review

State of the Art

In recent years, data-driven approaches have increasingly been applied to understand complex navigation patterns of users based on clickstream data in e-commerce contexts. Clickstream data (the ordered sequence of user clicks that represent a interaction with a website) provides valuable behavioral information that can be used to personalize recommendations, increase engagement, and improve conversion rates. As a result, clickstream analysis has become a central component of modern web analytics and recommender systems, offering insights into user behavior, preferences, and browsing habits.

Evolution of Clickstream Analysis Approaches

Mining Sequential Patterns (Agrawal & Srikant, 1995):

One of the earliest works introducing this concept of understanding complex navigation patterns of user based on clickstream data in e-commerce context was Mining Sequential patterns paper by Agrawal and Srikant in 1995 [2]. In this paper, the authors introduced algorithms for extracting frequent sequences from transactional data, laying the groundwork for sequence-aware analyses in diverse domains including clickstream mining.

A Survey of Sequential Pattern Mining (Fournier-Viger & Chung Wei, 2017):

Traditional pattern mining methods have evolved significantly with the standardization of the internet and the growth of online commerce and online purchasing, shifting from static pattern discovery toward approaches capable of handling dynamic, high-volume web interaction and clickstream data, as reviewed in *A Survey of Sequential Pattern Mining* [3].

BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformers (Sun et al., 2019):

In this work, the authors argue that unidirectional sequential models such as GRU [4] and LSTM [5] restrict the expressive power of hidden representations in user behavior sequences and often rely on a strictly ordered sequence assumption, which is not always practical in real-world recommendation scenarios. To overcome these limitations, they propose BERT4Rec [6], a sequential recommendation model based on deep bidirectional self-attention [7].

To prevent information leakage while enabling bidirectional modeling, the authors adopt a Cloze objective [6], where random items in the sequence are masked and predicted by jointly conditioning on both their left and right contexts. This training strategy allows the model to learn rich bidirectional representations, enabling each item in the user's historical behavior to incorporate information from both past and future interactions.

Extensive experiments conducted on four benchmark datasets demonstrate that BERT4Rec [6] consistently outperforms several state-of-the-art sequential recommendation models, highlighting the effectiveness of bidirectional self-attention for modeling user preferences.

PinnerFormer: Sequence Modeling for User Representation at Pinterest *(Pancha et al., 2022):*

In this study, the authors observe that while sequential models such as Gated Recurrent Units (GRU) [4] and Long Short-Term Memory (LSTM) [5] networks are effective for predicting users' next actions, they are often difficult to deploy in large-scale production environments. In

particular, such models typically require streaming infrastructures to continuously incorporate the latest user interactions with the website.

To address these challenges, the authors introduce PinnerFormer [8], a transformer-based user representation model trained to predict a user's recent actions. Instead of focusing on next-action prediction, PinnerFormer [8] reformulates the modeling objective to operate within a batch-processing infrastructure by introducing a dense all-action loss that captures long term future user behavior. This design bridges the gap between batch-generated user embeddings, which are typically updated periodically, and real-time user embeddings that are updated after each user interaction.

The proposed approach was deployed at Pinterest, where it led to measurable improvements in user retention and engagement compared to previous user representation models.

A Deep Learning Approach for Online User behavior Prediction Using Clickstream Data (Batool et al., 2023):

In this study, the authors introduce deep learning architectures based on Long Short-Term Memory (LSTM) [5] recurrent neural networks to predict online user behavior from clickstream data. Multiple LSTM architectures were developed and evaluated on their ability to accurately predict user actions like search, add to the cart and purchase. The proposed LSTM models achieved an overall accuracy of 85% in predicting user behavior sequences and individual actions. This work demonstrates the effectiveness of deep learning for modeling temporal patterns in clickstream data to gain insights into customer behavior [9].

TRACE: Transformer-based User Representations from Attributed Clickstream Event Sequences (Black et al., 2024):

As a recent advancement in this field, *TRACE* proposes a novel approach that enables machine learning models to learn complex user navigation patterns from sequential clickstream data. This represents a significant advancement, as the architecture builds upon the self-attention mechanism introduced in *Attention Is All You Need* [7] to model long-range dependencies in user behavior. *TRACE* introduces a transformer-based framework specifically designed to generate rich user embeddings from live, multi-session clickstream data for real-world recommendation applications. Through extensive experiments conducted on large-scale travel e-commerce datasets containing real user journeys, the authors demonstrate that *TRACE* outperforms vanilla transformer and large language model (LLM)-style architectures [1] .

Available datasets

This research project was conducted using a number of e-commerce datasets that are suitable for the *TRACE* architecture. In this section, we describe the types of datasets which are compatible with *TRACE* and explain the criteria used for their selection. To ensure the appropriate choice of datasets, several requirements were defined:

Requirements

- **E-commerce domain:** The dataset must originate from an e-commerce platform, as *TRACE* was designed to model and predict user behavior in this type of website.

- **Clickstream event sequences:** The dataset must consist of sequential clickstream events with associated timestamps, where each event represents a user navigation action. This sequential structure captures user behavior over time and enables the model to learn both contextual and temporal patterns within user interactions.

- **Suitability for multi-task learning (MTL):** The selected datasets must contain a variety of user clickstream actions, resulting in diverse user behavior patterns. This diversity of actions enables the application of multi-task learning (MTL), as *TRACE* is designed to jointly predict multiple user engagement signals from heterogeneous interaction data.

- **Large-scale datasets:** The datasets must be sufficiently large to enable the learning of meaningful user embeddings. A large volume of interaction data allows the *TRACE* model to capture diverse navigation patterns and generalize user behavior across different contexts and sessions.

- **Session-driven datasets:** The datasets must be session based and include user identifiers, allowing interactions to be grouped by individual users. This data structure is essential for *TRACE*, as the model is designed to learn consistent navigation patterns from the same users across multiple sessions and over time.

- **Long temporal horizons and sufficiently long sessions:** The datasets must contain user sessions that extended over time and include a sufficient number of click events. A large

temporal span between the first and last timestamp ensures that the model can learn meaningful sequential patterns, while filtering out very short sparse sessions helps prevent the model from learning noise from limited user interactions.

Data Sets:

OTTO Dataset: The OTTO session dataset is a large-scale, industry grade dataset designed to bridge the gap between academic research and real-world applications in session based and sequential recommendation [10]. It contains anonymized behavioral logs collected from the **OTTO Swiss retail company** that offers a wide range of products, including food, furniture, sporting goods, and cosmetics, through its web shop and mobile application.

This dataset supports both single-and multi-objective learning through tasks such as predicting clicks, add-to-cart events and orders(purchase), making it well suited for evaluating the *TRACE* architecture under different learning settings.
This dataset is the primary dataset used in this research project.

Data Size

- 12M real-world anonymized user sessions
- 220M events, consisting of clicks, carts and orders
- 1.8M unique articles in the catalogue

Data Structure

- **Session (int64):** The unique session id

- **Events:** The time ordered sequence of events in the session
    - **Aid (int64)**: The article id (product code) of the associated event
    - **Ts (int64)**: The unix timestamp of the event
    - **Type (int64) :** The event type, i.e., whether a product was clicked, added to the user's cart, or ordered during the session.

The OTTO dataset is provided in the *train.jsonl* format, which was originally released for a Kaggle competition. As illustrated in Figure 1, the data is organized at the session level, where each session is identified by a unique session ID.

Within each session, a sequence of clickstream events is stored, with each event containing an article identifier (*aid*), a timestamp, and an event type indicating the nature of the interaction (e.g., click, add-to-cart, or purchase).


```
{
    "session": 42,
    "events": [
        { "aid": 0, "ts": 1661200010000, "type": "clicks" },
        { "aid": 1, "ts": 1661200020000, "type": "clicks" },
        { "aid": 2, "ts": 1661200030000, "type": "clicks" },
        { "aid": 2, "ts": 1661200040000, "type": "carts"  },
        { "aid": 3, "ts": 1661200050000, "type": "clicks" },
        { "aid": 3, "ts": 1661200060000, "type": "carts"  },
        { "aid": 4, "ts": 1661200070000, "type": "clicks" },
        { "aid": 2, "ts": 1661200080000, "type": "orders" },
        { "aid": 3, "ts": 1661200080000, "type": "orders" }
    ]
}
```


Figure 1: Data format of the OTTO sessions stored in *train.jsonl*, where each session contains a unique session ID and a list of events. Adapted from *the Otto Github dataset repository* [10]

To evaluate a model's ability to predict future behavior, as required for deployment in a real-world web shop, the authors use a time-based validation split. The training set includes user session from a 4-week period, while the test contains sessions from the following week. To prevent information leakage, any training sessions that overlap temporally with the validation or test period are trimmed, ensuring a strict separation between past and future data. As illustrated in Figure 2, the OTTO dataset is temporally split into four weeks for training and one week for validation and testing.

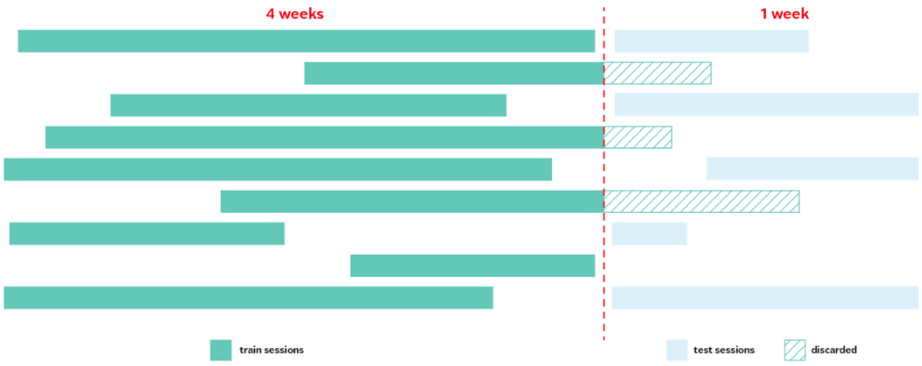


Figure 2: Temporal split of the OTTO dataset: four weeks for training and one week for validation and testing. Adapted from the *Otto Github dataset repository* [10]

For the action of clicking in the otto e-commerce website there is only a single ground truth value for each session, which is the next aid clicked during the sessions. The ground truth for carts and orders contains all aid values that were added to a cart and ordered respectively during the session.

Overall, the OTTO dataset offers an appropriate combination of scale, temporal structure, and multi-objective labeling that aligns well with the architectural design and learning objectives of

TRACE, making it a robust and realistic environment for evaluating the effectiveness of transformer-based user embedding models.

Synerise Dataset: The Synerise dataset is based on user click behavior that was collected from real-world online retailer's website, covering a period of six months. In the context of the RecSys Challenge (2025), the dataset **features Universal Behavioral Profiles** [11], which are user representations designed to encode essential aspects of an individual's historical interactions.

These profiles are designed to be universally applicable across multiple predictive tasks, such as churn prediction and product recommendation, making the dataset suitable for evaluating models like *TRACE* that aim to learn general-purpose user representations.

Data Size

- 160 million real-world anonymized user logs
- 150 million events corresponding to page visits
- 5.2 million events corresponding to add-to-cart actions
- 1.7 million events corresponding to product purchases
- 1.7 million events corresponding to remove-from-cart actions

Data Structure

The Synerise dataset consists of five types of events stored in Parquet file format, with each file containing a specific category of user interaction data. The event files include:

- ***product_buy.parquet***: Records whether a user purchased an item, including the numeric identifier (*sku*) of the purchased product.
- ***add_to_cart.parquet***: Records whether a user added an item to the shopping cart, including the numeric product identifier.
- ***remove_from_cart.parquet***: Records whether a user removed an item from the shopping cart, including the numeric product identifier.
- ***page_visit.parquet****:* Contains the pages visited by users, where each record includes the URL identifier of the visited page.
- ***search_query.parquet****:* Contains search queries performed by users, with each query represented as a compressed embedding vector corresponding to the search content at a given timestamp.

For the ***product_buy.parquet***, ***add_to_cart.parquet***, and ***remove_from_cart.parquet*** files, the data structure is as follows:

- **client_id (int64)**: Numeric identifier of the client (user).
- **timestamp (datetime)**: Timestamp of the event in the format YYYY-MM-DD HH:mm:ss.
- **sku (int64)**: Numeric identifier of the item (product).

For the ***page_visit.parquet*** file, the data structure is defined as:

- **client_id (int64)**: Numeric identifier of the client (user).

- **timestamp (datetime)**: Timestamp of the event in the format *YYYY-MM-DD HH:mm:ss.*
- **url (int64)**: Numeric identifier of the visited URL. Explicit information about the item presented on a given page is not provided.

For ***search_query.parquet*** the data structure is like this:

- **client_id (int 64)**: Numeric ID of the client (user).
- **Timestamp (object)**: Date of event in the format *YYYY-MM-DD HH:mm:ss.*
- **query (object):** The textual embedding of the search query compressed to 20-dimensional integer array using the product quantization method

The dataset has a continuous period of six months, enabling the modeling of long-term user navigation behavior and temporal dependencies.

The different events types like page visits, add-to-cart, purchase, remove-from-cart and search query are mapped to multiple prediction targets, enabling the application of *TRACE* architecture in a multi-task learning (MTL) setting.

Overall, the Synerise dataset provides a rich view of user behavior across multiple interaction types. Its structure and diversity of events make it well suited for evaluating *TRACE*.

Retail Rocket Recommender System Dataset:

The RetailRocket Recommender dataset was collected from a real-world e-commerce website. All client-related information was anonymized through hashing to preserve user confidentiality. The dataset contains clickstream interaction data, including events such as clicks, add-to-cart actions, and transactions, recorded over a period of approximately 4.5 months. In total, the dataset consists of 2,756,101 interaction events across 1,047,580 users [12].

Data Size:

- 2756101 anonymized interactions in total of 1047580 users in total
- 2664312 events corresponding to viewing items
- 69332 events corresponding to add-to-cart actions
- 22457 events corresponding to product purchases (transactions)

Data Structure

- **timestamp (int64):** Unix timestamp of the event (in milliseconds).
- **visitorId (int64):** Anonymous user identifier
- **eventType (string):** Type of interaction (e.g., view, add-to-cart, transaction)
- **itemId (int64):** Identifier of the product involved in the event.
- **transactionId (int64):** Identifier of the transaction (only present for *transaction* events; empty otherwise).

The different event type of interactions like add to the cart, visit a page and making a purchase(transaction) are mapped to multiple prediction targets, enabling the application of *TRACE* architecture in a multi-task learning (MLT) setting.

Overall, the retail rocket recommender systems provide a rich view of events of the users and the interactions with the e-commerce. The structure and diversity of events make it well suited for evaluating *TRACE.*

Methodology

Since the authors of *TRACE* paper did not publicly release the dataset used for training, validation and testing, it was necessary to identify alternatives that satisfy the requirements mentioned in the dataset selection criteria (To data selection criteria).

Selecting the Most Suitable Dataset

The first step of the methodology focused on reducing the sessions that were sparse in the user clickstream data. This was necessary because a significant number of *users or sessions* exhibited only a very small number of interactions events (clicks) (e.g., only land in the home page, click two parts of the homepage and leave)

Such short clickstream sequences are not informative for representation learning, as they do not capture meaningful navigation patterns or user intent. Including these sequences would negatively affect the quality of the learned user embeddings. Therefore, user with very limited clickstream events sequence were filtered out to ensure that the remaining data contained sufficiently rich behavioral signals for training the *TRACE* architecture.

This filtering was performed by applying a minimum threshold on the number of timestamps per session. Sessions or user that did not exceed this threshold were removed from the dataset. The initial threshold was set to 15 interactions events. This was done in order to analyze how the

dataset characteristics change as increasingly strict minimum interaction threshold is applied. **Table 1** summarizes the dataset changes after applying the interaction threshold, and **Figure 3** presents the corresponding distribution histograms.

Threshold = 15	Otto Dataset	Synerise Dataset	Retail Rocket Dataset
Total original sessions	12,899,779	22,298,361	2,756,101
Median (original)	6	2	1
Median after threshold	31	28	21
Sessions remaining after threshold	3,347,229	2,679,065	11,201
Data Eliminated for the threshold	0.7405	0.8799	0.9959

Table 1. Summary of dataset changes after applying a minimum interaction threshold to eliminate sessions with sparse clickstream events.

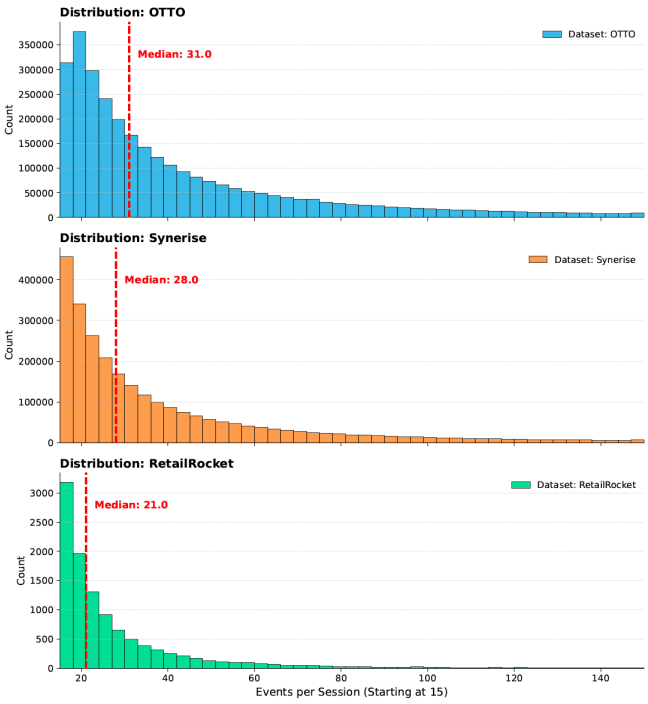


Figure 3. Histograms showing the dataset distributions after the first interaction threshold was applied.

OTTO Dataset

After applying the minimum interaction threshold to the OTTO dataset, a substantial number of session with sparse in clickstream events were eliminated. The original median session length increased from 6 to 31 clickstream events per session after filtering.

In terms of data volume, the number of sessions was reduced from 12,899,799 to 3,347,229, corresponding to a removal of approximately 74.05% of the original data.

Synerise Dataset

After applying the minimum interaction threshold to the Synerise dataset, a substantial reduction in the sessions that were sparse in clickstream events can be observed. The original median session length increased from 2 to 28 clickstream events per session after filtering.

In terms of data volume, the number of sessions was reduced from 22,289,361 to 2,679,065, corresponding to the removal of approximately 87.99% of the original data. This indicates that a large proportion of sessions were extremely sparse, containing too few interaction events to be considered informative for learning meaningful user representations.

Retailrocket recommender system dataset

After applying the minimum interaction threshold to the Retail Rocket dataset, an extreme reduction in sparse clickstream sessions was observed. The original median session length increased from 1 to 21 interaction events, indicating that the majority of sessions in the original dataset consisted of only a single click event.

In terms of data volume, the number of sessions was reduced from 2,756,101 to 11,201, corresponding to the removal of approximately 99.59% of the original data. This substantial indicates that it is not well suited for evaluating the *TRACE* architecture.

Final Dataset Selection for TRACE Evaluation

After analyzing the impact of the minimum number of interaction threshold on data volume, the OTTO and Synerise datasets were identified as the most suitable options for this investigation.

This decision was made by the fact that the Rocket Recommendation dataset was excessively reduced after the threshold, resulting in insufficient data for robust representation learning. Although the Synerise dataset remained viable after the threshold, it contained fewer number of sessions that the OTTO dataset. The OTTO dataset provided more data as the primary dataset for this investigation.

MODEL ARCHITECURE

The *TRACE* architecture is built around a transformer encoder block consisting of multi-head self-attention layer with eight attention heads, followed by a position-wise fully connected feed-forward network (FNN) with an intermediate dimensionality of 128. Dropout and residual connections are applied around each sub-layer, followed by layer normalization,

The resulting sequence representation is passed through multiple task-specific output heads corresponding to the *TRACE* prediction tasks. This multi-task design enables the model to learn shared user representations while jointly predicting different future engagement signals, as proposed in the original *TRACE* paper [1].

As illustrated in Figure 4, the *TRACE* architecture is trained to generate user journey embeddings from input clickstream sequences and to predict multiple future engagement objectives in a multi-task learning setting.

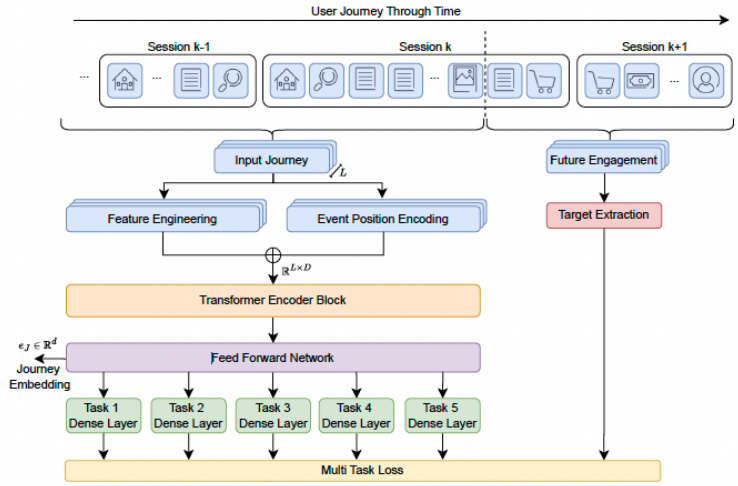


Figure 4: Overview of the *TRACE* multi-task transformer architecture, illustrating the transformation of input clickstream journeys into shared user embeddings and multiple task-specific predictions [1].

The Multi Task Training Regime (MLT) is used that by jointly predicting a diverse set of user engagement signals, the model is encouraged to learn comprehensive and generalizable representations that can be effectively utilized across a variety of downstream applications, extending beyond just the task during training [1].

Tasks for the Multi-Task Learning *TRACE*:

In the original *TRACE* paper, the authors propose a multi-task learning framework composed of five binary classification tasks, each representing a potential future action of a user [1].

- **PW2:** Make any purchase within two weeks
- **BN5**: Bounce within next five pages, and following which relate to actions within rest of session
- **SRP:** Make a search for a product
- **PDP:** View a product details page
- **VUO:** View an upcoming order.

Each task head has its own class-weighted binary cross-entropy loss function. The overall objective is expressed as a linear combination of these task-specific losses. For a user journey with model prediction ŷ and true labels y, the loss is defined as:

$$\mathcal{L}(J, y) = -\sum_{k=1}^{5}[w_k * y_k * \log(\hat{y}_k) + (1 - y_k) * \log(1 - \hat{y}_k)] \quad [1]$$

Class weights w_k are computed as the reciprocal of the proportion of positives samples for each task k in order to account for task-specific class imbalance. They weighted tasks equally to encourage the model to develop features which generalize across each task [1].

Tasks for the Multi-Task Learning reimplementation of *TRACE*:

In this research investigation, a multi-task learning framework similar to *TRACE* is proposed for the OTTO dataset. The framework consists of multiple binary classification tasks, each modeling a potential future user action:

1. **ATC (Add-to-Cart frequency):** Binary target equal to 1 if the user adds products to the cart at least two times during a session, and 0 otherwise.
2. **SAT (Repeated item views):** Binary target indicating whether a user views the same article identifier (AID) at least four times within a single session, reflecting strong browsing interest.
3. **MAP (Make a purchase):** Binary target equal to 1 if the user purchases a product during a session, 0 otherwise.

Each task employs a class-weighted binary cross-entropy loss, consistent with the original *TRACE* formulation. The overall objective function is computed as the aggregation of the individual task losses and is implemented using binary cross-entropy with logits as:

$$\mathcal{L}(J, y) = -\sum_{k=1}^{3}[w_k * y_k * \log(\hat{y}_k) + (1 - y_k) * \log(1 - \hat{y}_k)]$$

To address the strong class imbalance present in the dataset, a positive class weighting strategy was applied using the PyTorch **BCEWithLogitsLoss** formulation [13].

For each task k, the positive class weight was computed as:

$$w_k = \frac{N_k^-}{N_k^+}$$

Where N_k^+ and N_k^- are the number of positive and negative samples for task *k*, respectively.

This weighting scheme increases the contribution of minority positive samples during training and improves the model's ability to detect rare target events.

Data Split for Inputs and Targets

To construct the model inputs and the corresponding targets, a prefix-suffix splitting strategy was applied at the session level. For each user session, a random split point was sampled between 70% and 80% of the session length.

The events occurring before the split point were used as the input sequence, while the remaining events (clicks) formed the target sequence. This cut ensures that the model observes a sufficiently long interaction of clickstreams of the user navigation while still retaining future events for prediction. As it is shown on the Figure 5

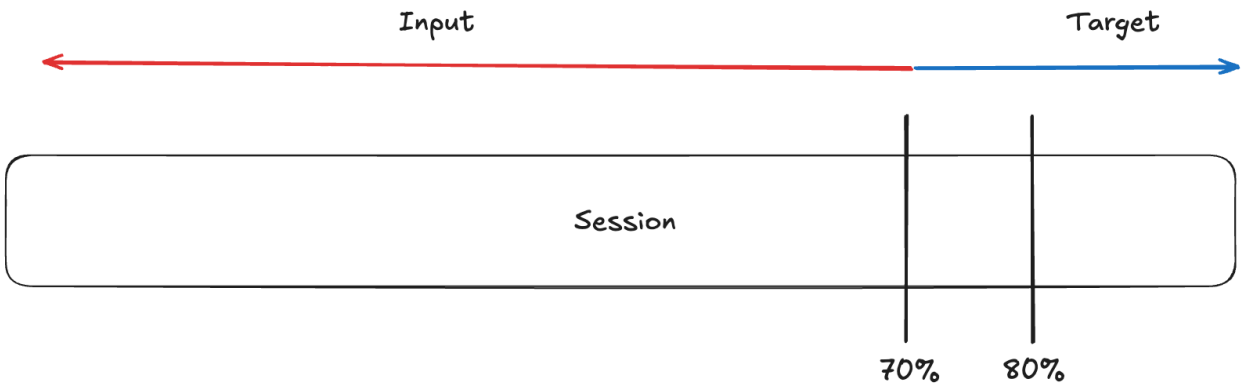


Figure 5. The split between input and target sequences was performed within a range of **70% to 80%** of each session length.

Final Threshold for Training, Validation and Testing:

The final threshold that was chosen was 32 interactions per session, meaning that sessions with fewer than 32 events were removed. This decision was made after performing data inspection and training experiments during the input–target splitting stage, where the model showed better

performance with this new threshold. In particular, the class balance between positive and negative samples improved.

As a result, the dataset size was reduced from 3,164,777 sessions to 1,621,909 sessions, while the mean number of clickstream events increased. The average input sequence length increased from 56.01 to 74.00, and the average target sequence length increased from 19.04 to 24.50. Consequently, the amount of padding required during training decreased and the number of informative target events per session increased, which contributes to more efficient learning and richer session representations a desirable property for sequential models like *TRACE*

Feature and Position Encoding

As a first step, each input session is cropped to retain at most the L most recent page view events, ensuring that the model captures the most relevant portion of the user's recent navigation history [1].

Each session in the OTTO dataset contains a set of categorical attributes, such as the article identifier (*aid*) and the event type (*type*). These attributes are passed through separate learnable embedding layers, which map the discrete inputs into dense vector representations in $\mathbb{R}^{32}$. These embeddings serve as the input feature representations for subsequent sequential modeling.

As a second step, two temporal features are engineered from the event timestamps: the time interval between consecutive events and the elapsed time since the most recent event. Both features are log-transformed and subsequently standardized.

All feature embeddings and engineered temporal features are concatenated to form the final input representation. As a result, each session is represented by a feature vector in $\mathbb{R}^D$, where D is on the order of a few hundred dimensions [1].

Finally, positional encoding is added to the input representation to provide the model with explicit information about the relative order of events within each session, enabling effective sequential modeling

For input sessions with length shorter than L, zero-padding is applied to the feature vectors to reach a fixed sequence length. The padding is done adding to zeros to the prefix of the sessions that were below the sequence length until it reached the sequence length

As a result, each session is represented as a matrix M $J \in \mathbb{R}^{L\ x\ D}$. where L denotes the maximum sequence length and D represents the dimensionality of the feature representation [1].

Experiments and Results

Comparable Models vs TRACE

The quality of the *TRACE* embeddings was evaluated against two comparable baseline approaches. The comparisons are reported as the mean performance uplift across all evaluation tasks. Results are shown in Table 2.

Single Task Cohort: To evaluate the effectiveness of TRACE's multi-task learning (MTL) approach, a dedicated single-task TRACE model was trained independently for each prediction task. The reported metrics correspond to the average performance across all tasks.

TRACE achieves a slightly higher AUPRC (0.696 vs. 0.694), indicating that the multi-task formulation improves ranking quality and prioritization of positive samples, which is particularly relevant under class imbalance conditions.

In contrast, the Single-Task Cohort model obtains marginally higher Binary F1 (0.664 vs. 0.662), Macro F1 (0.680 vs. 0.679), and Accuracy (0.694 vs. 0.689). These differences, although small, suggest that task-specific optimization may allow finer threshold calibration and class-specific tuning for individual objectives.

Both models achieve identical AUROC values (0.787), indicating comparable discriminative capability across classification thresholds. Overall, these results suggest that the main advantage of TRACE's multi-task learning lies in improved ranking performance (AUPRC), while single-task training remains slightly more effective for threshold-dependent metrics such as F1-score and accuracy.

MT Bi-LSTM: As shown in **Table 2**, the *TRACE* model outperforms the multi-task learning (MLT) Bi-LSTM baseline across all metrics. This result indicates that the transformer-based *TRACE* is more effective for capturing long-range dependencies and complex user navigation patterns in clickstream data

Model	AUROC	AUPRC	Binary F1	Macro F1	Acc
TRACE	**0.787**	**0.696**	0.662	0.679	0.689
ST Cohort	**0.787**	0.694	**0.664**	**0.680**	**0.694**
MT Bi-LSTM	0.738	0.664	0.620	0.661	0.679

Table 2.: *TRACE* and comparable models.

Evaluation of the Confusion Matrix

To evaluate the quality of the embeddings learned by *TRACE* in distinguishing between positive (1) and negative (0) classes, confusion matrices were generated for *TRACE* and the two baseline models. This section compares the resulting matrices to analyze classification behavior and error patterns.

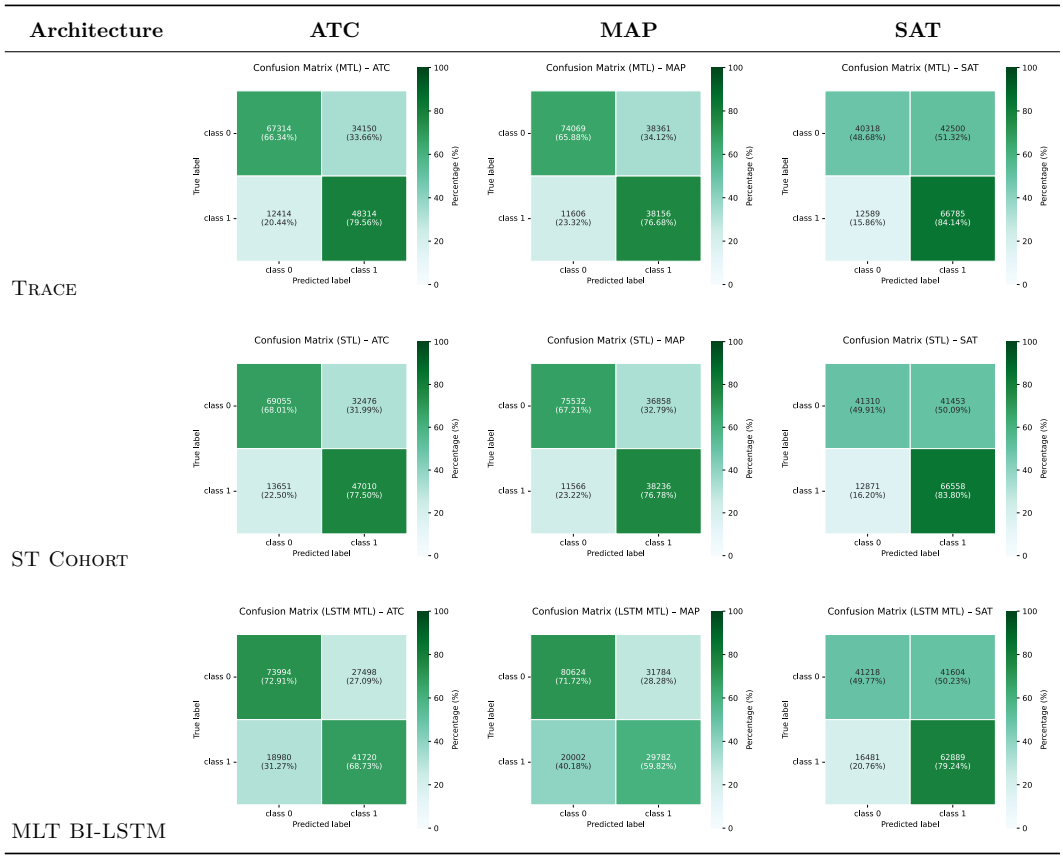


Figure 6.: Confusion Matrices obtained for the ATC, MAP and SAT tasks.

Confusion Matrix Comparison Between TRACE and Single Task Cohort

As shown in **Figure 6**, for the ATC (add-to-cart twice within a session) task, the *TRACE* multi-task learning (MTL) model achieves a higher true positive rate (79.59%) than the single-task cohort model (77.50%). Correspondingly, its false negative rate is lower (20.44%), indicating that the MTL model misses fewer positive (target) instances than STL (22.50%). However, MTL

exhibits weaker performance on the negative class, as reflected by a lower true negative rate (66.34%) and a higher false positive rate (33.66%) than STL, which achieves a higher true negative rate (68.01%) and a lower false positive rate (31.99%). This behavior suggests that while MTL is more effective at identifying positive events, it is also more prone to incorrectly classifying negative samples as positive.

For the MAP (purchase within session) task, the confusion matrix results in Table 3 indicate that the single-task learning (STL) model slightly outperforms the *TRACE* multi-task learning (MTL) model. Specifically, STL achieves a marginally higher true positive rate (76.78%) compared to MTL (76.68%) and a correspondingly lower false negative rate (23.22% vs. 23.32%). Moreover, STL shows stronger performance on the negative class, with a higher true negative rate (67.21% vs. 65.88%) and a lower false positive rate (32.79% vs. 34.12%). These results suggest that, for the MAP task, STL provides more balanced classification performance and is less prone to misclassifying negative samples as positive.

For the SAT (seeing the same item at least four times) task, the *TRACE* multi-task learning (MTL) framework achieves slightly better performance than the single-task cohort model in terms of positive-class detection. Specifically, MTL obtains a higher true positive rate (84.14%) compared to STL (83.80%) and a correspondingly lower false negative rate (15.86% vs. 16.20%), indicating improved detection of repeated-view events. However, MTL slightly underperforms on the negative class. The true negative rate is lower for MTL (48.68%) than for STL (49.91%), while the false positive rate is higher (51.32% vs. 50.09%). This indicates that although MTL is more effective at identifying positive SAT events, it is also more prone to incorrectly classifying negative samples as positive.

Overall, the results indicate that the MTL framework is more effective at predicting positive classes than the STL approach, outperforming it in two out of three tasks. However, when predicting negative classes, MTL shows a higher tendency to produce misclassifications, making it more prone to false positive errors.

Confusion Matrix Comparison Between TRACE and Multi-Task Learning Bi-LSTM

As shown in **Figure 6** for the ATC (add-to-cart twice within a session) task, the Bi-LSTM model is outperformed by the *TRACE* architecture in terms of positive-class detection. Specifically, *TRACE* achieves a higher true positive rate (79.56%) compared to Bi-LSTM (68.73%), and the false negative rate of (20.44% vs 31.27%). This indicates that the Bi-LSTM framework is substantially more prone to missing positive events.

In contrast, Bi-LSTM performs better on the negative class. It achieves a higher true negative rate (72.91%) than TRACE (66.34%) and a lower false positive rate (27.09% vs. 33.66%). This demonstrates that the Bi-LSTM model is more effective at correctly identifying negative samples (class 0). Overall, *TRACE* provides stronger performance for positive-class prediction, whereas Bi-LSTM exhibits better prediction of negative classes for this task.

For the MAP (purchase within a session) task, the Bi-LSTM model is clearly outperformed by the *TRACE* architecture in terms of positive-class detection. *TRACE* achieves a substantially higher true positive rate (76.68%) compared to Bi-LSTM (59.82%) and a correspondingly lower false negative rate (23.32% vs. 40.18%). This indicates that *TRACE* is significantly more effective at identifying purchase events.

In contrast, Bi-LSTM demonstrates stronger performance on the negative class. It achieves a higher true negative rate (71.72%) than *TRACE* (65.88%) and a lower false positive rate (28.28% vs. 34.12%). This behavior is consistent with the ATC task results, further confirming that Bi-LSTM is more effective at predicting negative classes, whereas *TRACE* provides superior performance for positive-class detection.

For the SAT (seeing the same item at least four times) task, the Bi-LSTM model is slightly outperformed by the TRACE architecture in terms of positive-class detection. TRACE achieves a higher true positive rate (84.14%) compared to Bi-LSTM (79.24%) and a lower false negative rate (15.86% vs. 20.76%), indicating improved detection of repeated-view events.

Regarding negative-class prediction, both models exhibit similar performance. The true negative rate is slightly higher for Bi-LSTM (49.77%) compared to *TRACE* (48.68%), while the false positive rate is marginally lower for Bi-LSTM (50.23% vs. 51.32%). This behavior can be partially attributed to the class imbalance and the weighting strategy applied during training, which biases the optimization process toward improving positive-class recall.

Overall, the experimental results demonstrate a consistent performance trade-off between the *TRACE* and Bi-LSTM architectures. TRACE systematically achieves higher performance on predicting positive behavioral signals across ATC, MAP, and SAT tasks, making it more effective at detecting meaningful user interactions such as purchases and repeated item engagement. In contrast, Bi-LSTM exhibits stronger conservative behavior, achieving higher true negative rates and lower false positive rates, which indicates better discrimination of non-target events.

Conclusion

In this study, the TRACE architecture was re-implemented, introducing a multi-task learning (MTL) framework based on a lightweight encoder-only transformer for generating user embeddings from multi-session clickstream sequences [1]. The experimental evaluation was conducted on the OTTO dataset, a large-scale real-world retail e-commerce dataset [10]. The results demonstrate that TRACE embeddings achieve stronger performance in predicting positive user engagement signals compared to the traditional Bi-LSTM-based model. For the MAP (purchase within a session) task, TRACE and the single-task cohort model exhibit comparable performance in positive-class prediction, while TRACE outperforms the single-task cohort model in two out of the three evaluated tasks. However, the *TRACE* architecture shows lower performance in predicting negative classes when compared to the Bi-LSTM and single-task cohort approaches.

This behavior may be explained by differences in the input–target splitting strategy. In the original *TRACE* paper, the authors do not specify a fixed split ratio, stating only that "*to generate our datasets, we split each journey at a random point and designate the pages before as the input journey, and those after to be used for target generation*" [1]. In contrast, this investigation applied a fixed random split range between **70%** and **80%** of the session length for the creation of input and target sequence length. This discrepancy may affect the proportion and distribution of target events available during training. As a result, *TRACE* may receive fewer effective negative samples in the target portion compared to the single-task cohort and Bi-LSTM models, which could partially account for the lower negative-class performance. In addition, the imbalance mitigation setup (as detailed in the methodology) may shift the decision boundary toward improved positive detection at the cost of negative-class discrimination.

Overall, these findings indicate that *TRACE's* multi-task formulation is particularly effective for ranking-oriented objectives and positive engagement prediction; however, its performance remains sensitive to dataset construction and training configuration. Future work should systematically evaluate alternative data splitting strategies and threshold calibration methods to further improve the robustness and generalization capability of the *TRACE* architecture.

References

[1] Black, W., Manlove, A., Pennington, J., Marchini, A., Ilhan, E., & Markeviciute, V. (2024). *TRACE: Transformer-based user representations from attributed clickstream event sequences*. arXiv. https://doi.org/10.48550/arXiv.2409.12972

[2] Agrawal, R., & Srikant, R. (1995). *Mining sequential patterns*. *Proceedings of the 11th International Conference on Data Engineering (ICDE)* (pp. 3–14). IEEE Computer Society. https://rsrikant.com/papers/icde95.pdf

[3] Fournier-Viger, P., Lin, J. C.-W., Kiran, R. U., Koh, Y. S., & Thomas, R. (2017). *A survey of sequential pattern mining*. *Data Science and Pattern Recognition, 1*(1), 54–77. https://www.philippe-fournier-viger.com/survey_sequential_pattern_mining.pdf

[4] Cho, K., et al. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proc. EMNLP* (pp. 1724–1734). https://arxiv.org/abs/1406.1078

[5] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

[6] Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W., & Jiang, P. (2019). *BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer*. arXiv. https://arxiv.org/abs/1904.06690

[7] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. arXiv. https://arxiv.org/abs/1706.03762

[8] N. Pancha, A. Zhai, J. Leskovec, C. Rosenberg, Pinnerformer: Sequence modeling for user representation at pinterest, in: Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining, 2022, pp. 3702–3712 https://arxiv.org/abs/2205.04507

[9] batool, iqra, A Deep Learning Approach for the Online User Behaviour Pediction Using Clickstream Data. Available at SSRN: https://ssrn.com/abstract=4662627 or http://dx.doi.org/10.2139/ssrn.4662627

[10] Normann, P., Baumeister, S., & Wilm, T. (2023). *OTTO Recommender Systems Dataset: A real-world e-commerce dataset for session-based recommender systems research* [Dataset]. Kaggle. https://www.kaggle.com/datasets/otto/recsys-dataset

[11] Synerise S.A. (2025). RecSys Challenge 2025 Dataset: Universal Behavioral Profiles [Dataset]. https://recsys.synerise.com

[12] Zykov, R., Noskov, A., & Anokhin, A. (2022). RetailRocket recommender system dataset [Dataset]. Kaggle. https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset

[13] PyTorch. (s. f.). *torch.nn.BCEWithLogitsLoss* — PyTorch documentation. https://docs.pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html